



Computer Graphics and Advanced Game Programming Topics Project 2

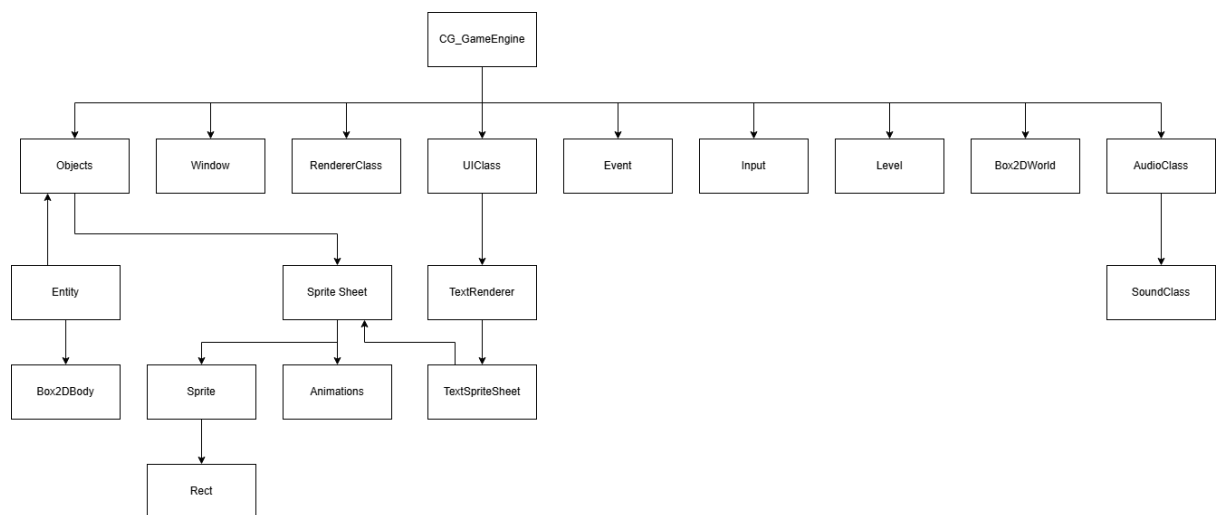
Miguel Balancho n.º 2221911

Report

Introduction

In this project I created a game engine using SDL3, OpenGL and Box2D as well as a horizontal copy of the game Xenon 2000 using said engine. I was able to achieve every objective asked for the project.

Class Diagram



Game Engine

The engine is built as a static library that uses SDL3 for window management and input, OpenGL for rendering, and Box2D for physics. The CG_GameEngine class controls all subsystems. When the engine starts, it initializes a window with OpenGL context, creates the rendering system, sets up input, creates the physics world, and starts the audio manager. The engine also manages the game loop, based on unity's execution order, where delta time is calculated and other classes are updated.

The Level class provides an interface with OnStart() and OnUpdate() functions that the in-game levels can inherit from.

```
while (isRunning)
{
    int now = SDL_GetTicks();
    deltaTime = (now - lastFrameTime) / 1000.0f;
    lastFrameTime = now;

    physicsWorld->FixedUpdate(deltaTime);

    events();

    currentLevel->OnUpdate(deltaTime);

    for (int i = objectArray.size() - 1; i >= 0; i--)
    {
        objectArray[i]->Update(deltaTime);

        if (objectArray[i]->GetToDestroy())
        {
            delete objectArray[i];
            objectArray.erase(objectArray.begin() + i);
        }
    }

    for (int i = UIArray.size() - 1; i >= 0; i--)
    {
        UIArray[i]->OnUpdate(deltaTime);
    }

    audioManager->Update();
    render();
}
```

Texture Loading

The Texture class handles loading BMP files and converting them into OpenGL textures. When a texture is loaded it uses SDL to read the file and convert it to rgba32 format. During this conversion any magenta pixels are made transparent by setting their alpha to zero.

```
SDL_Surface* rgbaSurface = SDL_ConvertSurface(surface, SDL_PIXELFORMAT_RGBA32);
SDL_DestroySurface(surface);

if (rgbaSurface == nullptr) { ... }

Uint8* pixels = static_cast<Uint8*>(rgbaSurface->pixels);
int pitch = rgbaSurface->pitch;
for (int y = 0; y < rgbaSurface->h; ++y) {
    Uint8* row = pixels + y * pitch;
    for (int x = 0; x < rgbaSurface->w; ++x) {
        Uint8* px = row + x * 4;
        Uint8 r = px[0];
        Uint8 g = px[1];
        Uint8 b = px[2];

        if (r == 255 && g == 0 && b == 255) {
            px[3] = 0;
        }
    }
}
```

After conversion the texture data is uploaded to the GPU using OpenGL. The texture is configured to use nearest filtering to stay pixel-perfect. Each texture stores its width and height so that sprite sheets can calculate the correct coordinates for individual sprites.

The rendering system uses OpenGL, replacing the SDL rendering from the previous project. It uses a single reusable quad that is scaled and positioned using transformation matrices to draw all sprites.

An orthographic projection matrix maps pixel coordinates to the OpenGL coordinate system with the origin at the top-left corner to make it work in the same way it does in SDL. Each sprite draw call sets up a model matrix containing the sprite's position, rotation, and scale, then passes texture coordinates to the shader to select the correct region of the sprite sheet.

The sprite shader uses two main uniforms: texOffset and texSize, which define which portion of the texture to display. This allows a single texture to contain multiple sprites while only rendering the requested region. The fragment shader also discards fully transparent pixels to handle sprite transparency correctly.

Spritesheets

The `SpriteSheet` class takes a texture and divides it into a grid of equally-sized sprites based on how many columns and rows the original file has. During initialization it calculates the width and height of each sprite by dividing the texture dimensions by the number of columns and rows. It then creates individual `Sprite` objects for each cell in the grid, storing them in a vector for easy access.

```
void SpriteSheet::CutSpriteSheet()
{
    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < columns; col++) {
            int x = col * spriteWidth;
            int y = row * spriteHeight;

            Sprite* sprite = new Sprite(texture, x, y, spriteWidth, spriteHeight);
            sprites.push_back(sprite);
        }
    }
}
```

Sprites can be retrieved either by index or by column and row coordinates. The `SpriteSheet` also supports creating custom sprites that don't align to the grid by specifying exact pixel coordinates and dimensions.

Each `Sprite` object holds a reference to its parent texture and stores a source rectangle defining which portion of the texture it represents. This rectangle is used during rendering to calculate the correct texture coordinates.

Objects

The Object class serves as the base for all visual game elements. When an object is created it automatically registers itself with the engine's object array. Objects can be created just with a reference to the engine or with a position, size and a sprite or spritesheet.

If an Object is created with a sprite sheet the object receives cloned copies of all animations defined in that sheet, allowing multiple objects to share the same sprite sheet while maintaining independent animation states.

Objects have an Update function that is called on the engine's loop that updates the animation and can be used by children of the object class to have logic run on update.

When an object calls *Destroy()* it sets a flag that the engine checks during its update loop.

I also created an Entity class that descends from Object here I added Box2D physics integration. When an entity creates a physics body it handles the conversion between pixel coordinates and the meter coordinates used by Box2D.

Animation

Animations are created using sequences of sprites. When an animation is created it receives a list of sprite indices from a sprite sheet, a frame duration, and whether it should loop.

```
Animation::Animation(std::string n, SpriteSheet* sheet, std::vector<int> indices, bool loop, float framelenght)
{
    name = n;
    frameTime = framelenght;
    isLooping = loop;

    PrepareFrames(sheet, indices);
}

void Animation::PrepareFrames(SpriteSheet* sheet, std::vector<int> indices)
{
    if (sheet == nullptr) return;

    for (int index : indices) {
        Sprite* sprite = sheet->GetSprite(index);
        if (sprite != nullptr) {
            frames.push_back(sprite);
        }
    }
}
```

The animation class uses delta time for in its update making it frame-rate independent. When Update() is called the animation accumulates time and advances to the next frame when enough time has passed. If the animation reaches the end it either loops back to the beginning or remains on the final frame.

```
void Animation::Update(float deltaTime)
{
    if (!isPlaying || frames.empty()) return;

    elapsedTime += deltaTime;

    if (elapsedTime >= frameTime) {
        elapsedTime = 0.0f;
        currentFrame++;

        if (currentFrame >= frames.size()) {
            if (isLooping) {
                currentFrame = 0;
            }
            else {
                currentFrame = frames.size() - 1;
                isPlaying = false;
            }
        }
    }
}
```

Input

The Input class provides keyboard and gamepad input. In order to make the input user friendly for usage in the game I created a namespace where I keep the scancode of each key, this way, when it's needed to check for a input in the game a function called *GetKey()* can be used as *GetKey(Key::W)*. This same logic also applies to controller input.

```
namespace Key {  
    const int A = 4;  
    const int B = 5;  
    const int C = 6;  
    const int D = 7;  
    const int E = 8;  
    const int F = 9;  
    const int G = 10;  
    const int H = 11;  
    const int I = 12;  
    const int J = 13;  
    const int K = 14;  
    const int L = 15;  
    const int M = 16;  
    const int N = 17;  
    const int O = 18;  
    const int P = 19;  
    const int Q = 20;  
    const int R = 21;  
    const int S = 22;  
    const int T = 23;  
    const int U = 24;  
    const int V = 25;  
    const int W = 26;  
    const int X = 27;  
    const int Y = 28;  
    const int Z = 29;  
}
```

The Event class wraps SDL's event system providing a cleaner interface for handling window events, input events, and controller connections. During each frame the engine polls for events and routes them to the appropriate systems. This includes quit events when the window is closed, keyboard and gamepad input events, and window resize events.

Box 2d

The physics system uses Box2D 3.0 for collision detection. A Box2DWorld object manages the physics simulation and is updated with a fixed timestep to ensure consistent physics behavior regardless of frame rate.

Entities create physics bodies as sensors which detect overlaps without applying physical forces. These bodies are created using *CreatePhysicsBody()*, this function can be called empty or with values for width and height of the physic body, if it is created empty it will automatically use the entities width and height. When two of these bodies overlap the engine calls the *OnBeginOverlap()* method on both entities.

UI

The UI system consists of a UIClass which is used as the base class for any UI that is created in the game, here it is possible to create TextRenderers to display text or UI objects. These UI objects are managed separately from game objects and have their own update and render loops ensuring they're always drawn on top of the game world.

In order to display text using a spritesheet I created a TextSpriteSheet class that extends from SpriteSheet. Here I create a map that keeps a reference to what the position of each character should be in the spritesheet. This way when a character is requested this class will look up its index in the character map and return the corresponding sprite.

The TextRenderer takes a TextSpriteSheet and provides methods for rendering text at specific positions. It supports basic text rendering, centered text, and multi-line text with configurable character spacing and line height. Text is rendered by drawing individual character sprites in sequence, calculating their positions based on the specified coordinates and spacing.

Sound

The sound system is built on SDL3's audio functionality and consists of the SoundClass for individual sounds and the AudioClass for managing playback.

When a sound is loaded the SoundClass reads the corresponding WAV file and stores the audio data and format information. Since different WAV files can have different sample rates, bit depths, and channel counts the sound is converted to match the audio device's output format. This conversion happens once during loading so that playback is efficient.

The AudioClass manages the audio device and handles mixing multiple sounds. When a sound is played a new audio stream is created and bound to the device. SDL3 automatically mixes all bound streams allowing multiple sounds to play simultaneously without manual mixing code.

When a sound is played the system creates a stream, queues the sound data, and tracks the stream until it finishes playing. Finished streams are cleaned up during the Update() method to free resources.

Game

Now, for the game itself, I only ever include the header for the game engine. On the main of the game I create the Engine, set the level and start the engine loop.

The level is a child of the level class from the engine but I only use the OnStart() function from there. Here I start by creating all the fonts, spritesheets and animations that will be needed for the different objects as well as the sounds that will play throughout the game, then I set a background image, start the music using one of the sounds and create the player, a spawner that handles the enemies and a background layer to produce the parallax effect.

The background layer is an object that creates a number of custom sprites that are saved in a vector, then those sprites are slowly moved left until they go out of the window. When this happens they are moved back to the right of the window and keep looping like this in order to create the parallax effect.



Player

The Player is a class that descends from Entity, it has its own health, can move around with user input, has multiple animations and shoots missiles.



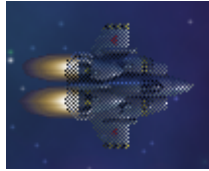
When the player is initialized it gets a reference to the input from the engine, there is a *CheckInput()* function that is called every update, in here the player detects if the wasd keys are pressed and moves accordingly, it also checks when space is called and calls a *Shoot()* function. The *CheckInput()* function also detects if there is a controller connected, if there is it can use the left analog stick for the player movement and the lower face button to shoot. Before applying the movement there is a check to make sure the player is within the bounds of the screen, if it is close to the border the player is stopped from moving in that direction.

While the player is moving there are two variables that keep track of the direction it is moving in, these are also used in *ManageAnimations()* function that is called every update, in here the player will do a different turn animation depending on the direction it is currently moving. Before playing any animation this function verifies what animation was playing previously in order to set the current animation to one that fits into the previous one.



If the previously mentioned *Shoot()* function is called the player will first check if it can shoot. This is determined by a variable that is changed in a *CalcShootCD()* function which is called every update, in there I have a variable that keeps track of the time since the player last shot, if that time is larger than the cooldown that is set when the player is initialized then the player will be allowed to shoot. If *Shoot()* is called and the player can shoot then there is a missile entity that is created, this entity moves to the right and explodes when it collides with something, dealing damage if it is an enemy.

The player starts with three lives, whenever its health reaches zero one of these lives is removed and the player loses any power ups it had. The player becomes invulnerable to damage for a small period of time while playing an animation that makes them transparent. When the player's lives reach zero the game is closed.



When the player is created in the Level it is given a reference to an object that represents its boosters, this object's position is updated every player update to keep it attached to the player, Its size is also constantly being changed to give it an animation that makes it look like it's pulsating.

Power Ups

The Power Ups are entities the player can pick up by colliding with them to receive benefits. There are three types of power ups.

First I created a damage and health power up. These can be spawned when an enemy dies. When the player picks up a health power up it receives a small amount of health, this health can heal it beyond its original max health. When the player picks up a damage power up the strength of its missiles is raised making them have a different sprite and deal more damage.



The last power up is the companion, this type of power up is spawned using the same spawner that spawns the enemies, one of these spawns every twenty seconds. When the player receives this power up a small ship appears floating beside it, the player can have up to two companions at the same time. The companions are always attached to the players and shoot whenever the player does. They can pick up the other two power ups and have their own health. If a companion takes enough damage it dies and the player can pick up a new one.

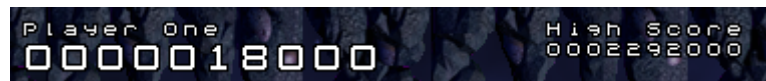


UI

The in-game UI descends from the *UIClass*. Here I create text renderers for the score and high score as well as sprites for the player's lives and current health.

Score is added when the player kills an enemy, when this happens the enemies score is added to the score variable that is kept in the UI. When an enemy dies there is also a text that appears in the place of the enemy showing how much score they gave to the player, these texts are kept in a vector and are destroyed after a couple of seconds.

To keep track of the high score I created a .txt file where I store the current score when the game is closed, if it is bigger than the previous high score. When the game is loaded the UI will get the high score from that file and show it in a text renderer.



To show how many lifes the player has I created a `SetLives()` function, in here I create a number of sprites corresponding to the amount of lives the player has, this is called in the beginning of the game and set to three. Then it is called every time the player's health reaches zero.

The player's current health is represented by a bar on the bottom left of the screen. To create this bar I added a file to the project with different colors to represent the state of the player's health. The bar starts with a pre determined length and is colored as green, every time the player takes damage the length of the bar is updated and if the player loses enough health it's color changes to either yellow or red. If the player gains health over its original limit a second bar appears over the original one colored blue.



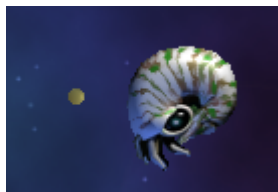
Enemies

In order to create the different enemy types I created an enemy class from which the other enemies would descend from. In this class I have the function for when the enemies take damage as well as an *OnBeginOverlap()* that deals damage to the player when they collide. When an enemy dies I use a *rand()* to make it so that there is a ten percent chance that it spawns a power up.

To handle the enemy spawning I created a Spawner object, this one doesn't have any sprite. Inside this object I have a timer running that makes it so that every couple of seconds one of the enemies is spawned randomly using a *rand()*. When the enemies are spawned I made it so that their position is also randomly chosen, since this random number has to have a range i use a *random_device* instead of a *rand()*.

Then I created the different enemies that descended from the main Enemy class, these are four types of enemies: Loner, Rusher, Drone and Asteroids.

The Loner enemy moves sideways and shoots at the player, to make the movement i give the enemy a limit to close to the top and bottom of the screen then make it so that it keeps moving back and forth between those limits. In order to make it shoot I gave it a similar code to the player shooting, every couple seconds the Loner will shoot a missile that deals damage when it collides with the player.



The Rusher enemy is even simpler, these can only damage the player if they collide with it. I always spawn these enemies at the right of the screen and make it so that they move towards the left. If they get out of the screen without dying I make sure to destroy it so that the game doesn't need to calculate its movement and animations when they aren't going to be seen again.



The Drone enemy works similarly to the rusher however they also move up and down as go towards the left of the screen. To make them move up and down I give them a limit on how high and low they can go, when they spawn they start moving up and when they reach this limit they start moving in the opposite direction. These enemies spawn in groups and their movements make a pattern similar to a wave, to make this effect I made it so that it has a small interval between spawning each enemy from the group.



The Asteroids are also counted as enemies, these also work very similarly to the rusher, they float to the left of the screen, their main difference is that when a rock is destroyed it spawns three smaller asteroids, these are given a direction so that they move in a fan pattern from where they were destroyed. The asteroids can also be metallic, if that is the case then they can't be destroyed.



References

SDL migration -

<https://github.com/libsdl-org/SDL/blob/main/docs/README-migration.md>

uniform int distribution -

https://en.cppreference.com/w/cpp/numeric/random/uniform_int_distribution.html

SDL documentation -

<https://wiki.libsdl.org/SDL3>

Box2D documentation -

<https://box2d.org/documentation/index.html>

Sounds used -

music - <https://freesound.org/people/Romariogrande/sounds/370801/>

ship shooting - <https://freesound.org/people/LilMati/sounds/459145/>

ship losing a life - <https://freesound.org/people/SamsterBirdies/sounds/467882/>

enemy shooting - <https://freesound.org/people/hotpin7/sounds/819682/>

pick up - <https://freesound.org/people/qubodup/sounds/140773/>

rock breaking - <https://freesound.org/people/iwanPlays/sounds/567250/>

explosion - <https://freesound.org/people/Duskbreaker/sounds/641486/>